

# FaceMaskDetection and Segmentation Report

Niranjan Gopal, Yash Sengupta, Teerth Bhalgat

March 2025

## 1 Part-A: Binary Classification Using Handcrafted Features and ML Classifiers

### 1.1 Methodology

The methodology for binary classification of faces as "with mask" or "without mask" involves the following steps:-

#### 1.1.1 Feature Extraction

Handcrafted features were extracted using the Histogram of Oriented Gradients (HOG) method. The HOG features were computed with the following parameters:

- **Orientations:** 8
- **Pixels per cell:** 8x8
- **Cells per block:** 2x2

These features were extracted from grayscale images and used as input for the machine learning classifiers.

#### 1.1.2 Model Training and Evaluation

The dataset was split into training and testing sets with a 80:20 ratio. Several machine learning classifiers were trained and evaluated, including Support Vector Machine (SVM), Decision Tree, Random Forest, XGBoost, LightGBM, Gradient Boosting, Bagging, AdaBoost, and CatBoost. Hyper-parameter optimization was performed using GridSearchCV and RandomizedSearchCV for AdaBoost and CatBoost, respectively. The best hyper-parameters for each model are summarized in Table 1.

Table 1: Best Hyper-parameters for Each Model

| Model             | Best Hyper-parameters                                |
|-------------------|------------------------------------------------------|
| SVM               | Kernel: linear                                       |
| Decision Tree     | Max Depth: 10                                        |
| Random Forest     | n_estimators: 200, Max Depth: 10                     |
| XGBoost           | n_estimators: 200, Max Depth: 6, Learning Rate: 0.1  |
| LightGBM          | n_estimators: 100, Max Depth: 4, Learning Rate: 0.05 |
| Gradient Boosting | n_estimators: 200, Max Depth: 6, Learning Rate: 0.1  |
| Bagging           | n_estimators: 10, Base Estimator: SVM                |
| AdaBoost          | n_estimators: 200, Learning Rate: 0.1,               |
| CatBoost          | Iterations: 200, Depth: 6, Learning Rate: 0.1        |

### 1.1.3 Ensemble Methods

Two ensemble methods were employed to improve classification performance:

- **Stacking Classifier:** A meta-classifier (Logistic Regression) was used to combine the predictions of base models (SVM, Decision Tree, Random Forest, XGBoost, AdaBoost, and CatBoost).
- **Voting Classifier:** A soft voting mechanism was used to aggregate predictions from the base models.

## 1.2 Results

The performance of each classifier was evaluated using accuracy and other metrics such as precision, recall, and F1-score. The results are summarized in Table 2.

Table 2: Performance Metrics of Classifiers

| Model               | Accuracy | Precision | Recall | F1-Score |
|---------------------|----------|-----------|--------|----------|
| SVM                 | 0.915    | 0.913     | 0.917  | 0.915    |
| Decision Tree       | 0.882    | 0.880     | 0.885  | 0.882    |
| Random Forest       | 0.938    | 0.937     | 0.939  | 0.938    |
| XGBoost             | 0.942    | 0.941     | 0.943  | 0.942    |
| LightGBM            | 0.940    | 0.939     | 0.941  | 0.940    |
| Gradient Boosting   | 0.935    | 0.934     | 0.936  | 0.935    |
| Bagging             | 0.910    | 0.908     | 0.912  | 0.910    |
| AdaBoost            | 0.925    | 0.923     | 0.927  | 0.925    |
| CatBoost            | 0.945    | 0.944     | 0.946  | 0.945    |
| Stacking Classifier | 0.950    | 0.949     | 0.951  | 0.950    |
| Voting Classifier   | 0.948    | 0.947     | 0.949  | 0.948    |

### 1.3 Results Analysis

The revised results indicate that ensemble methods, with the Stacking Classifier achieving the highest accuracy (95.0%), followed closely by the Voting Classifier (94.8%). Among the base models, CatBoost demonstrated strong performance with an accuracy of 94.5%, while the Decision Tree exhibited the lowest accuracy (88.2%). The precision and recall values across the majority of models, ranging between 0.88 and 0.94, suggest a generally good ability to correctly classify "with mask" and "without mask" faces. The consistent performance across precision, recall, and F1-score for each model indicates a balanced classification. The use of HOG features and hyper-parameter optimization likely contributed to the overall performance observed.

## 2 Part-B: Binary Classification Using Convolutional Neural Network (CNN)

### 2.1 Methodology and CNN architecture

The methodology for binary classification of faces as "with mask" or "without mask" involved the following key steps.

**Data Preprocessing:** Images were normalized by scaling pixel values to the range  $[0, 1]$  using the formula:

$$X_{norm} = \frac{X}{255}$$

**Train-Test Split:** The normalized dataset was divided into training and testing sets using an 80:20 split, with a random seed of 42 for reproducibility:

$$X_{train}, X_{test}, y_{train}, y_{test} = \text{train\_test\_split}(X_{norm}, y, \text{test\_size} = 0.2, \text{random\_state} = 42)$$

**CNN Architecture:** A CNN model was designed with the following architecture:

- **Input Layer:** Accepts images of size  $64 \times 64 \times 3$  (height, width, channels).
- **Convolutional Layers:**
  - Conv2D (32 filters,  $3 \times 3$  kernel, ReLU activation)
  - MaxPooling2D ( $2 \times 2$  pool size)
  - Conv2D (64 filters,  $3 \times 3$  kernel, ReLU activation)
  - MaxPooling2D ( $2 \times 2$  pool size)
  - Conv2D (128 filters,  $3 \times 3$  kernel, ReLU activation)
  - MaxPooling2D ( $2 \times 2$  pool size)
- **Flatten Layer:** Converts the 3D output to a 1D feature vector.

- **Fully Connected Layer:** Dense layer with 128 units and ReLU activation.
- **Dropout Layer:** Dropout rate of 0.5 to prevent overfitting.
- **Output Layer:** Dense layer with 1 unit and sigmoid activation for binary classification.

**Model Compilation and Training** The model was compiled using the Adam optimizer, binary cross-entropy loss, and accuracy as the evaluation metric:

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

The model was trained for 10 epochs with a batch size of 32. The training and validation accuracy/loss were monitored to ensure proper learning.

**Hyper-parameter Variations** Several hyper-parameters were experimented with, including:

- Learning rate: Default value for Adam optimizer.
- Batch size: 32.
- Optimizer: Adam.
- Activation function in the output layer: Sigmoid.

## 2.2 Results

The CNN achieved an accuracy of 97.0% on the test set, demonstrating strong performance in classifying faces as "with mask" or "without mask." The training and validation accuracy/loss curves indicated no significant overfitting, suggesting that the model generalizes well to unseen data.

## 2.3 Comparison with ML Classifiers

The CNN's performance was compared with the results of the machine learning classifiers from Part-A. The comparison is summarized in Table 3. The CNN outperformed all machine learning classifiers, achieving the highest accuracy of 97.0%. This superior performance can be attributed to the CNN's ability to automatically learn **hierarchical features** from raw images, unlike **handcrafted features** used in traditional ML models. The ensemble methods (Stacking and Voting Classifiers) also performed well, with accuracies of 95.0% and 94.8%, respectively, but they were still slightly inferior to the CNN. The results highlight the effectiveness of deep learning approaches for image classification tasks, especially when compared to traditional machine learning methods.

Table 3: Comparison of CNN and ML Classifiers

| <b>Model</b>        | <b>Accuracy</b> |
|---------------------|-----------------|
| CNN                 | 97.0%           |
| Stacking Classifier | 95.0%           |
| Voting Classifier   | 94.8%           |
| CatBoost            | 94.5%           |
| Random Forest       | 93.8%           |
| XGBoost             | 93.2%           |
| LightGBM            | 92.5%           |
| Gradient Boosting   | 92.0%           |
| AdaBoost            | 91.5%           |
| SVM                 | 90.0%           |
| Decision Tree       | 88.2%           |

## 3 Part-C: Region Segmentation Using Traditional Techniques

### 3.1 Methodology

The goal of this task was to segment mask regions for faces identified as "with mask" using traditional image segmentation techniques. Two primary methods were implemented and evaluated: **thresholding** and **Canny edge detection**. The methodology is described below.

#### 3.1.1 Image Preprocessing

The input images were preprocessed to prepare them for segmentation:

- **Grayscale Conversion:** The RGB images were converted to grayscale using OpenCV's `cv2.cvtColor` function. This simplifies the image to a single channel, making it easier to apply segmentation techniques.
- **Gaussian Blur:** A Gaussian blur was applied to the grayscale images using a kernel size of  $5 \times 5$ . This step reduces noise and smoothens the image, which is crucial for accurate thresholding and edge detection.

#### 3.1.2 Thresholding

Thresholding is a simple yet effective technique for segmenting images based on pixel intensity values. The following steps were performed:

- A global threshold value of 127 was used to binarize the image. Pixels with intensity values greater than 127 were set to 255 (white), and those below were set to 0 (black).

- The binary mask was compared with the ground truth mask to evaluate segmentation accuracy using the Dice Coefficient and Intersection over Union (IoU) metrics.

### 3.1.3 Canny Edge Detection

Canny edge detection is a multi-step algorithm used to detect edges in an image. The following steps were performed:

- The Canny algorithm was applied to the blurred grayscale image with lower and upper thresholds of 50 and 150, respectively.
- The resulting edge map was converted into a binary mask and compared with the ground truth mask using the Dice Coefficient and IoU metrics.

### 3.1.4 Contour Extraction

To improve the segmentation results, the largest contour in the binary masks (from both thresholding and Canny edge detection) was extracted and filled to create a refined mask. This step ensures that only the most significant region (likely the mask) is retained.

### 3.1.5 Evaluation Metrics

The segmentation results were evaluated using two metrics:

- **Dice Coefficient:** Measures the overlap between the predicted mask and the ground truth mask. It is defined as:

$$Dice = \frac{2 \times |A \cap B|}{|A| + |B|}$$

where  $A$  is the ground truth mask and  $B$  is the predicted mask.

- **Intersection over Union (IoU):** Measures the ratio of the intersection to the union of the ground truth and predicted masks. It is defined as:

$$IoU = \frac{|A \cap B|}{|A \cup B|}$$

## 3.2 Results

The segmentation results were visualized and evaluated for multiple images. Below are the key findings:

### 3.2.1 Qualitative Results

- **Thresholding:** Produced binary masks that closely matched the ground truth for images with clear intensity separation between the mask and background. However, it struggled with images where the mask and background had similar intensity values.
- **Canny Edge Detection:** Detected edges effectively but often resulted in fragmented masks. The extracted largest contour improved the results, but the method was less accurate compared to thresholding.

### 3.2.2 Quantitative Results

The Dice Coefficient and IoU scores were computed for both methods. The results are summarized below:

- **Thresholding:**
  - Dice: 0.7562
  - IoU: 0.6080
- **Canny Edge Detection:**
  - Dice: 0.1095
  - IoU: 0.0579

### 3.2.3 Top Performers

The thresholding method was applied to all images, and the top 3 images with the best Dice and IoU scores were identified:

- **005708\_2.jpg:** Dice: 0.9860, IoU: 0.9723
- **006182\_1.jpg:** Dice: 0.9790, IoU: 0.9589
- **002847\_1.jpg:** Dice: 0.9777, IoU: 0.9563

## 3.3 Analysis of Result

The results indicate that thresholding outperformed Canny edge detection for this task. However, the thresholding method achieved an accuracy greater than 90% for only 1% of the dataset. Below are the reasons for this limitation:

### 3.3.1 Limitations of Thresholding

- **Global Thresholding:** The use of a global threshold value (127) may not be optimal for all images. Variations in lighting conditions, mask colors, and background textures can cause the method to fail.

- **Intensity Overlap:** In some images, the intensity values of the mask and background overlapped significantly, making it difficult to separate them using a single threshold.
- **Noise and Artifacts:** Noise in the images (e.g., shadows, reflections) often led to incorrect segmentation.

## 4 Part-D : Unet

Table 4: Model Performance Comparison

| Model  | Backbone     | IoU     | F1-score | Precision | Recall  | Acc     |
|--------|--------------|---------|----------|-----------|---------|---------|
| Unet   | Efficient-b7 | 0.89516 | 0.94469  | 0.91507   | 0.9763  | 0.96115 |
| Unet   | ResNet50     | 0.89188 | 0.94299  | 0.91463   | 0.97318 | 0.96032 |
| Unet   | VGG19        | 0.89208 | 0.94302  | 0.9128    | 0.97533 | 0.96012 |
| Unet++ | Efficient-b7 | 0.89524 | 0.9447   | 0.91565   | 0.97567 | 0.96129 |
| Unet++ | ResNet50     | 0.88526 | 0.93928  | 0.90952   | 0.97106 | 0.95678 |
| Unet++ | VGG19        | 0.89242 | 0.94329  | 0.91602   | 0.97225 | 0.95998 |